may retrieve irrelevant passages, causing the model to answer with high confidence and low accuracy. An agentic workflow may call the wrong tool, loop unnecessarily, or exceed latency budgets. Without observability, these failures often surface as vague complaints from users rather than precise engineering signals. Teams spend time guessing which part of the pipeline is responsible, and improvements become slow and expensive.

Observability is equally important from an operational and financial perspective. LLM applications are not only compute-intensive -- they are also usage-sensitive. Small changes in prompt length, retrieval depth, model choice, or conversation memory can materially alter token consumption and therefore cost. End-to-end tracing makes these shifts visible. It also helps organisations enforce governance by showing which model was invoked, what context was supplied, whether sensitive data appeared in prompts or outputs, and how long each step took. For teams operating under security, privacy, or compliance constraints, this visibility is not a convenience. It is a prerequisite for responsible deployment.

Another reason observability matters is that AI systems improve through iteration. Teams constantly refine prompt templates, swap models, tune retrieval strategies, introduce safety filters, and redesign agent workflows. Each change can improve one dimension while degrading another.

Key signals to monitor in AI and LLM workloads

Useful observability for AI systems begins with collecting the right signals. At the request level, teams typically monitor latency, throughput, success and failure rates, retry behaviour, and time spent in downstream dependencies such as vector databases or external APIs. At the model level, token counts, prompt size, completion size, provider metadata, model version, and estimated cost are essential. In RAG architectures, retrieval-specific signals matter just as much: document recall, ranking quality, chunk relevance, source coverage, and the relationship between retrieved context and final response. In agentic systems, tool invocation paths, decision sequences, fallback behaviour, and loop detection become critical.

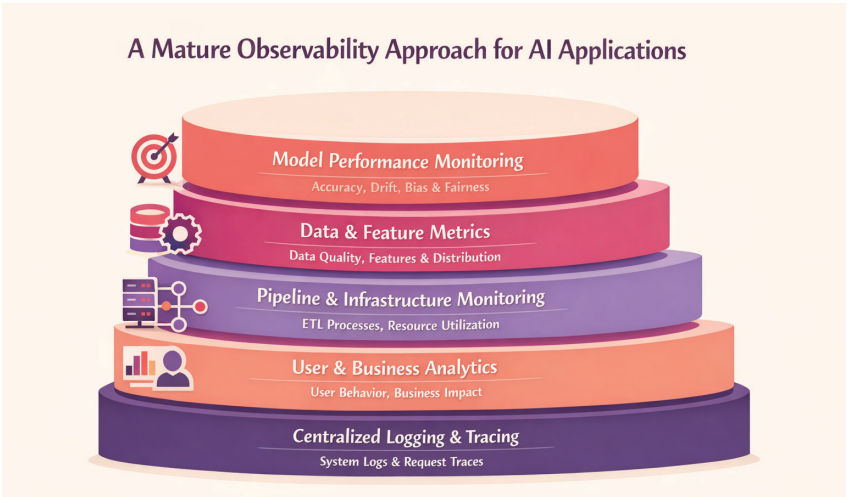

Figure 1: A multi-layered observability approach

Operational telemetry alone is not enough. AI applications also require quality and safety signals. These may include groundedness, factual consistency, relevance, response coherence, refusal behaviour, harmful content detection, prompt injection indicators, policy violations, and user feedback scores. Many teams also track business-aligned outcomes such as task completion, deflection rate, conversion impact, or escalation frequency. The real strength of AI observability comes from linking these qualitative measures with technical telemetry. When a drop in relevance can be tied to a prompt revision or a retrieval latency spike, root cause analysis becomes far more actionable.

Open source observability tools for AI and LLM applications

The open source ecosystem for AI observability has evolved rapidly. Some tools focus on instrumentation and standards, others on trace visualisation, prompt analytics, evaluation workflows, or cost tracking. The strongest implementations often combine more than one component: a standard telemetry layer, a trace collection pipeline, and a specialised interface for AI-specific analysis. Choosing among them depends on architecture, privacy requirements, engineering maturity, and whether the team prefers SDK-based integration, proxy-based capture, or a hybrid model.

OpenTelemetry and OpenLLMetry: OpenTelemetry has become one of the most important foundations for AI observability because it offers a vendor-neutral way to collect traces, metrics, and logs across distributed systems. For AI and LLM workloads, this matters because model calls rarely live in isolation. They are part of larger workflows that include web applications, APIs, retrieval services, orchestration frameworks, caches, databases, and external tools. By instrumenting AI flows with OpenTelemetry, teams can connect model behaviour with the rest of the application stack instead of operating two separate monitoring worlds.

OpenLLMetry, from the Traceloop ecosystem, builds on OpenTelemetry